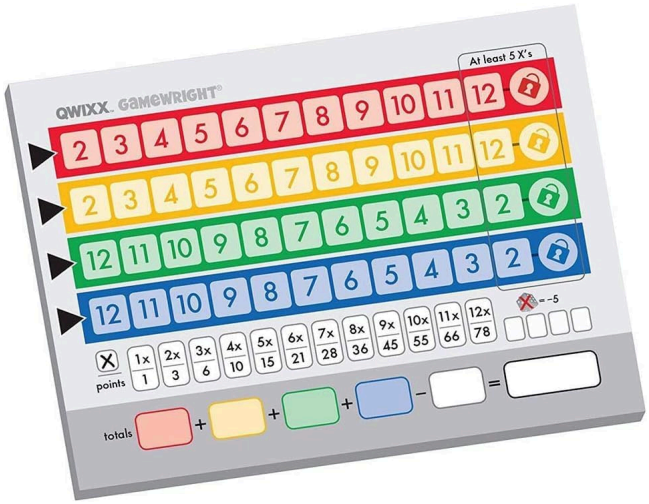
Final Project: A Reinforcement Learning Approach to Quixx

Ben Gonzalez

Introduction



Quixx is a fast-paced dice game for 2-5 players, ages 8 and up. It has sold millions of copies and remains a fixture of family game nights, at least in circles I am aware of. On every turn, the active player rolls two white dice and four colored dice (red, yellow, green, and blue). All players may immediately cross off the sum of the two white dice in any one of their four colored rows, provided the number appears to the right of any marks already made. The active player has an additional option to also cross off the sum of one white die and one matching colored die in the corresponding row. Marks must always proceed sequentially (red and yellow ascend from 2 to 12; green and blue descend from 12 to 2). A row locks once its final number is crossed off, removing that colored die from future rolls. The game ends when any two rows are locked or when a player fills all four penalty boxes (*bottom-right corner of the scorecard image above*). Scoring follows a triangular pattern per row (n marks result in the nth triangular number: 1, 3, 6, 10, ..., 66), with a 5-point deduction per penalty. The final score is the sum across all rows minus penalties.

The core challenge is making optimal sequential decisions under uncertainty. Dice rolls introduce stochasticity, while every choice weighs the immediate scoring opportunities against future row-completion potential. For example, you may avoid taking a box that skips 2 or 3 numbers, fearing that the low odds of rolling an 11/12/2/3 on your next turn would result in taking a penalty. There is also the consideration of whether other players are close to locking a row, or how many crosses you already have in any given row. Human players typically rely on rough heuristics like avoiding taking penalties early or not skipping more than three or four boxes. But, it is unclear whether these rules of thumb are actually optimal. The same decision structure appears in real-world settings across organizations and industries as parties decide how to manage inventory, balance investment portfolios, and more. Quixx with reinforcement learning (RL) gives some insight into optimizing these types of decisions.

<u>This project addresses the question:</u>

Can RL – specifically Q-Learning, Monte Carlo Tree Search (MCTS), and Deep Q-Network (DQN) – discover effective policies for Quixx, and what strategic insights if any do these policies reveal?

My approach was to build a custom Gymnasium environment that simulates the rules of Quixx, and train three RL agents. The key finding is that **DQN was the only method that**

outperformed a random baseline, achieving an average final score of +6.92 (random baseline was +1.69). Considering some limitations, like the tendency of these agents to take penalties, that likely means a score before penalties of close to 30 points. This shows that a simple deep reinforcement learning agent can discover strategies superior to randomness in a real-world game. This project also exposes some of the practical limits of Q-Learning and MCTS under constraints like computation and time. This opportunity provided a practical way to explore gameplay in this family game while applying RL to state spaces with sequential decision making.

Approach

Quixx is a great opportunity to simulate and experiment with reinforcement learning. It provides a Markov Decision Process (mathematical framework) for modeling sequential decision-making under some random conditions. It does not require an external dataset, though having data on human gameplay would be very useful for supervised learning or more advanced reinforcement learning. The project instead generates its own gameplay data through simulation. I created two **Python modules** for this project to work effectively in a Python Notebook.

- **quixx_game.py** contains logic for the complete rules (dice rolls, legal moves, sequential marking, locking, penalties, and scoring), including nuances for order of specific turns like two players locking two rows on the same turn.
- **quixx_env.py** wraps the game as a Gymnasium environment compatible with standard RL libraries.

The observation space is a 58-dimensional integer vector with 48 entries for the two players' scorecards (4 rows * 11 mark positions plus flags for locks, tracking penalties, number of locked rows, which player is active, and the six current dice values). The action space is a MultiDiscrete([5, 9]) that encodes the legal combinations of white-sum and colored-die marks on the scorecard, with illegal moves being filtered out at each action. This guarantees that agents can never propose invalid actions. Early in the analysis, this was an issue as agents would make illegal moves in order to avoid taking penalties.

The data-generation approach is sensible for this question. Simulating gameplay allows us to gain millions of state spaces and games in minutes, and without any data on Quixx gameplay it was necessary in this instance. It captures the stochasticity of the game with random dice rolls and dice sums that follow the same normal distribution the physical dice would, as well as the sequential constraints the scorecard introduces. It also captures the same **scoring function** that drives this project question, where the score for a row with (n) crosses follows a triangular number pattern, and the total score is dependent on the number of crosses in each row (r, y, g, b) and number of penalties p:

$$S(n) = \frac{n(n+1)}{2}$$

$$Score = S(r) + S(y) + S(g) + S(b) - 5p$$

Some **limitations** include the fact that this version of the project is constrained to 2 players. While Quixx is usually played in groups up to 5, this model focuses solely on a two-player version. Expanding it to 3, 4, or 5 would conceivably be similar, but with more potential for programming in

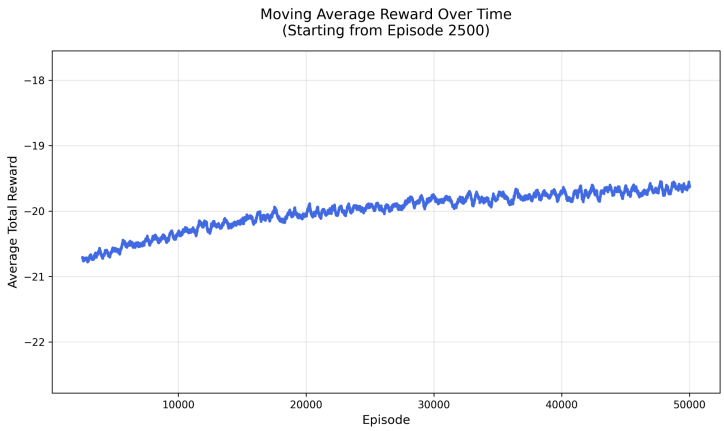
factors like broader game state considerations (e.g., if other players are close to locking multiple rows). Another limitation is that we cannot fully capture all of the elements of competitive play with a simulated version of the game. There are nuances like psychological factors or errors that cannot be reflected in the data. Like most analyses, data is a shadow of what we really care about, but simulating the data gives us some sense for ways we can learn to optimize Quixx. These Python functions also use some simplified encoding that discards long-term history (MCTS, which is future-looking, has limitations on how far down its decision tree it can explore; DQN has a limited capacity).

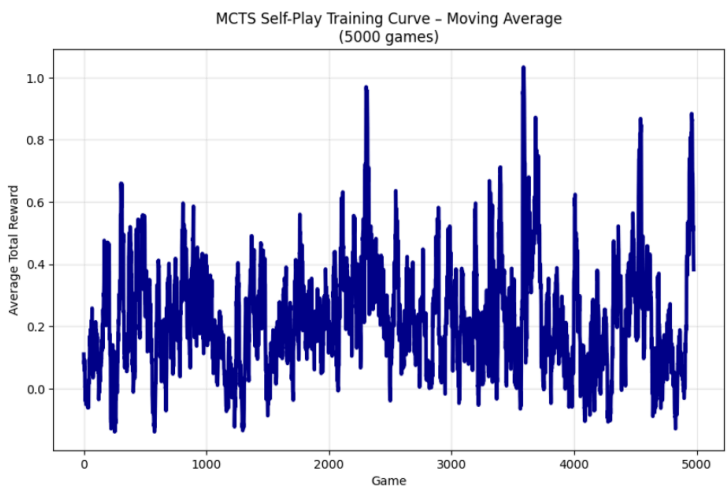
Three reinforcement learning methods were implemented and compared under identical conditions (the same reward function and game logic). Q-Learning stores Q-values in a dictionary keyed by flattened observation tuples. It updates with a learning rate, discount, and epsilon controlling exploration and exploitation (the agent's tendency to try new strategies versus sticking with the best known strategy). Monte Carlo Tree Search uses Upper Confidence Bound selection as a hyperparameter, with 300 simulations per move. Deep Q-Network uses a three-layer MLP (58 ⟶ 128 ⟶ 128 ⟶ 45 actions) with experience replay that has a capacity of 10,000 and a batch size of 64. This means, it can keep up to 10,000 past experiences, and during training, it randomly samples 64 experiences at a time. DQN uses two networks, a main network that learns constantly and a target network that is copied from the main network every 200 steps. This prevents the model from destabilizing learning. DQN has a similar epsilon hyperparameter for exploration and exploitation.

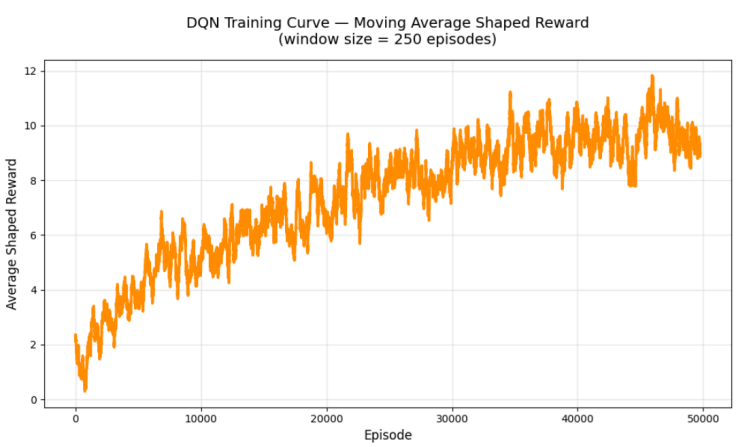
Each model received the same shaped reward:

- +(new_score - old_score) for the score gained during the turn
- +0.1 if the agent successfully made a legal move
- +(final score * 0.05) as a terminal reward when the game ends
- Penalties are reflected in the score difference

This design aligns with Quixx's non-linear scoring system while discouraging aimless play. However, as I will explain later in this analysis, this was definitely a scoring function I could improve on in the future. I chose deep learning, specifically DQN, because the state space of Quixx is enormous. In an iteration where I spent just 1,000,000 training episodes on a Q-Learning model, the output was a Q-table that explored 8.5 million potential states. Tabular methods like Q-Learning cannot generalize across similar states, but neural networks provide the necessary ability to generalize. I included MCTS because I wanted to explore another approach, and read about its involvement in training AlphaGo. This gave me a search-based algorithm to compare Q-Learning and DQN. In the analysis, I trained Q-Learning and DQN for 50,000 episodes and MCTS for 5,000 episodes (due to a more tedious computation requirement). The code in the provided IPYNB and environment files can be used to effectively reproduce the findings below.







Analysis & Results

Training revealed drastic differences in learning dynamics and overall performance. **Q-Learning** showed slow but steady improvement, after 50,000 episodes improving the moving-average shaped reward from roughly -20.6 to -19.6 (*see blue line*). The post-training evaluation showed an average Quixx score of -6.38 across 1,000 test games. Q-Learning performed below random, showing how difficult it is to capture a discrete large state space with simple RL. The Q-Learning agent visited roughly 450,000 unique states, but still defaulted to taking penalties in many situations.

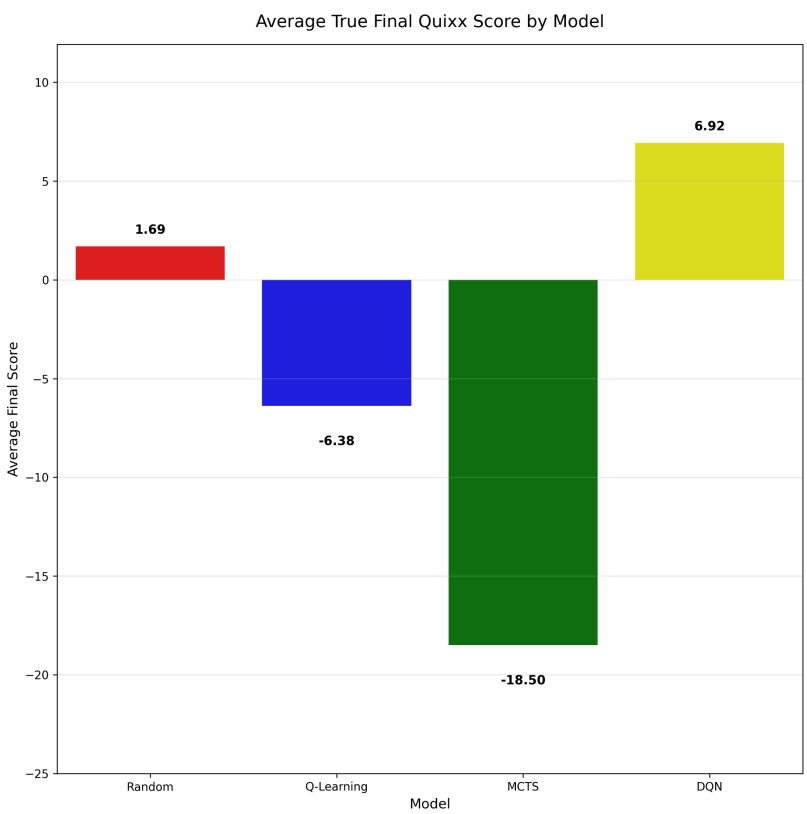
MCTS, despite multi-agent (self-play) learning and observing forward states, produced very variable rewards that hovered around 0.2 with no clear upward trend. Its average score on 200 evaluation games was -18.5, severely underperforming the random baseline of +1.69. Both non-deep methods struggled with the large state space and limited training scope (*see purple line*).

Deep Q-Network, by contrast, learned rapidly and effectively. Its shaped-reward curve rose steadily over its 50,000 training episodes, beginning to plateau near +10 after around 40,000 episodes. On 1,000 evaluation games, the DQN achieved an average reward of +8.2 and an average Quixx game score of +6.92. This was more than four points better than the random baseline, which is negligible compared to human gameplay but shows real learning occurring. The model generalized well across unseen states. While the learning curve for MCTS is noisy and flat, and the curve for Q-Learning is gradual but with a very poor learning intercept, DQN shows clear improvement and a more promising trajectory (*see orange line*).

A head-to-head comparison with 1,000 test games for the Random Baseline, Q-Learning and DQN – and 200 test games for MCTS – confirms which models were most valuable:

Model	# of Training Episodes	# of Test Games	Avg Reward	Avg Quixx Score	Avg Penalties Taken
Random (Baseline)	--	1,000	2.62	1.69	4
Q-Learning	50,000	200	-6.00	-6.38	4
Monte Carlo Tree Search	5,000	1,000	-19.29	-18.50	4
Deep Q-Network	50,000	1,000	**8.20**	**6.92**	4

The most surprising and disappointing outcome was that all four agents averaged a maximum total of 4 penalties taken per game, indicating need for improvement in the current reward system. While this reward framework was better than some earlier iterations I tried, it clearly needs to punish taking penalties more, reward actions that result in more points, or more training to explore state spaces where taking penalties is less optimal. I think there are ways to implement multi-agent training as well that would overcome this hurdle, through combining DQN and MCTS or other methods.



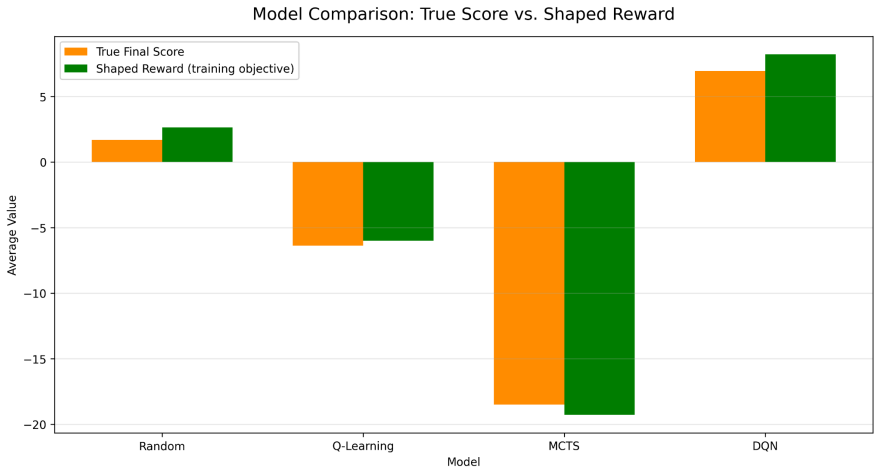
Nonetheless, the Deep Q-Network had the most successful average Quixx score, and also averaged the highest episode reward score. In terms of training time for this task, and how successful each model was, I was most impressed with the DQN's network, and disappointed with the performance of MCTS. It could be a fault in the way I applied it, but I was hoping MCTS would interpret the non-discrete state space more effectively than simple Q-Learning. MCTS was the most computationally expensive, and with better context or some combination of supervised learning could be useful.

Discussion, Conclusion, and Next Steps

The results effectively answered my project question. Yes, reinforcement learning (particularly Deep Q-Network) can discover policies superior to random gameplay in Quixx. These policies have the capability to reveal insights that human heuristics may overlook. One simple strategy that Q-Learning picked up on is playing the combination of a colored die and white die on the first active turn. From a

simple statistical perspective, this makes perfect sense. The likelihood of the two white dice summing to an optimal starting move (2 or 12; or even 3 or 11) is lower than either of the two white dice summing with any of the colored dice for one of those numbers. Because the state space is so large for Quixx, it was more difficult to find other interpretable game states, but in the future I would love to explore other opening sequences like the ideal move when a player's best option is to take a penalty or skip 3 or 4 squares. Q-Learning also demonstrated being more selective about the white dice when inactive (e.g., choosing to utilize the white dice less when there was no penalty at stake). These insights were very interesting, but the overall performance gap between Q-Learning and DQN highlights why deep function approximation is essential. Tabular and tree-search methods could not scale as well to Quixx's large state space. Especially under tight time constraints for this project.

Besides limitations I mentioned earlier like a two-player model, limited training time, and the penalty-heavy behavior that all of the agents exhibited, there is room for improving how the game state is represented at a vector level, including how agents observe one another's scorecards. There is also a lot of room to tune the reward function I used, and while I tuned some of the hyperparameters for each model, I'm sure there are more optimal ways to tune them as well. This project is a large simplification of how Quixx is played, and while the models came close to a random baseline, I have a great appreciation for the human brain and its ability to reason and balance short-term gains with long-term opportunities – even in the context of a game.

To conclude, this project shows that a straightforward deep reinforcement learning agent can definitely extract non-trivial strategies from a dice game, turning what is normally a casual game into a laboratory for making better decisions and reasoning to optimize a reward or score. DQN's result showed me that RL can create a reproducible improvement over pure chance, and that this project has a strong foundation for deeper study.

Next steps for this project include adding full self-play for the DQN (removing the random opponent that is included in the game logic), experimenting with other reward functions to reduce the average penalties taken, extending from two players to 3 or more, and increasing the scale of training the models. I think that with even a few extra hours of training, there would be drastic improvement on top of the training I did for each model in just thirty minutes or less. The ultimate goal for this project would be to deploy the best agent as an opponent a human could interact with, which would be a means to collect more data for this project and continue to improve the best model. That would also close the loop between this simulation and my lived experience playing and enjoying this game. This project has the power to change how I play Quixx, and with some simple improvements and training time, it could change how others play it as well.